CSAI 205

Fundamentals of Circuits and Electronics

Potentiometer Controlled Robotic Arm Using Arduino Uno

With Color Sensing

Prepared by: Yasmin Badran

Student ID: 202505432

Instructor: Assoc. Prof. Mohamed Maher Ata

Teaching Assistant: Ahmed Bilal

1. Components

Component	Quantity
Arduino Uno	1
Servo Motors	4
Potentiometers (50 kOhm)	4
TCS34725 Color Sensor	1
External 5 V DC Power Supply	1

The arm is controlled manually. Each potentiometer controls one servo. The color sensor measures light from an object near the gripper. Its reading does not start automatic arm movement.

2. Operating

• The user turns a potentiometer. Its centre terminal produces a changing voltage.

• Arduino reads that voltage through an analog input.

• The code filters the reading and converts it to an angle within the selected range.

• Arduino sends a control signal to the matching servo.

• The sensor measures red, green, blue and clear light levels through I2C.

• The code compares the reading with the background and reference colors.

• Serial Monitor displays Angle1, Angle2, Angle3, Angle4 and Color.

Power: The external 5 V supply powers the motors. Arduino 5V powers the potentiometers. The sensor VIN is connected to Arduino 3.3V, as used in the actual project. All grounds share one reference.

3. Explanation: Arduino Uno

Arduino Uno is a microcontroller board based on the ATmega328P. It reads inputs, runs a stored program and controls outputs. In this project, the inputs are four potentiometers and a color sensor. The outputs are servo control signals and Serial Monitor text.

Main parts

Part	Simple explanation
ATmega328P	The main chip that executes the uploaded program.
USB connector	Connects to the computer for uploading, serial communication and board power.
USB-to-serial interface	Connects the computer serial link to the main microcontroller.
Reset button	Restarts the program. It does not erase the uploaded code.
Barrel power jack	An alternative board power input. It is not the servo signal connector.
16 MHz clock	Provides the timing reference used by the microcontroller.
Power regulator	Helps provide the board supply voltage from a suitable external input.
ON / TX / RX / L LEDs	Show power, serial activity, and the built-in LED connected to D13.

An analog signal can change over a range of voltages. A digital signal uses HIGH and LOW levels. A servo control signal is a sequence of timed digital pulses; it is not the motor power supply.

Arduino Uno: Pins and Their Functions

Pin or group	Function and use in this project
D0 (RX)	Receives UART serial data. Used by the USB serial connection; not connected to a servo here.
D1 (TX)	Transmits UART serial data. Used for Serial Monitor output.
D2, D3, D4, D5	Digital input/output pins. This code uses them as servo signal outputs.
D6, D7, D8, D9	General digital input/output pins. Unused in this project.
D10, D11, D12, D13	Digital input/output pins that also support SPI: SS, COPI/MOSI, CIPO/MISO and SCK. No SPI sensor is used here. D13 also connects to the built-in LED.
Pins marked ~	D3, D5, D6, D9, D10 and D11 support analogWrite() PWM. Servo.h can also generate servo pulses on D2 and D4, which do not have the ~ mark.
A0, A1, A2, A3	Analog inputs. Each reads one potentiometer wiper.
A4 / SDA	Analog input with an I2C data function. Used as SDA for the sensor.
A5 / SCL	Analog input with an I2C clock function. Used as SCL for the sensor.
SDA and SCL header pins	Duplicate the same I2C signals as A4 and A5 on the Uno R3. They are not a second I2C bus.
5V	Board 5 V supply pin. Powers the potentiometers in this project.
3.3V	Regulated 3.3 V output. Connected to sensor VIN in this project.
GND	Ground reference shared by the circuit and external motor supply.

Pin functions are taken from the official Uno pinout. Project assignments are selected in the submitted code.

Arduino Uno: Remaining Pins and Connections

Pin or connector	Meaning
VIN	External supply input to the Arduino board regulator. This is different from sensor VIN. The Uno normally uses 7-12 V for recommended external board input; do not confuse this with a motor 5 V rail.
RESET	Pulling this signal LOW resets the main microcontroller.
IOREF	Indicates the board I/O operating voltage to compatible shields. On Uno R3, the I/O operates at 5 V.
AREF	An external analog reference input used with the proper analogReference() configuration. Left unused in this sketch.
NC	Not connected. It is not a spare power pin.
ICSP header	Provides programming/SPI connections, including power, ground and reset. It is not used by this project.

Why these project pins? A0-A3 provide four separate analog readings. D5-D2 provide four separate servo signal outputs. A4 and A5 are used because the sensor needs the Uno I2C data and clock lines. The exact four servo pins are a code choice, not a requirement that every arm must use these pins.

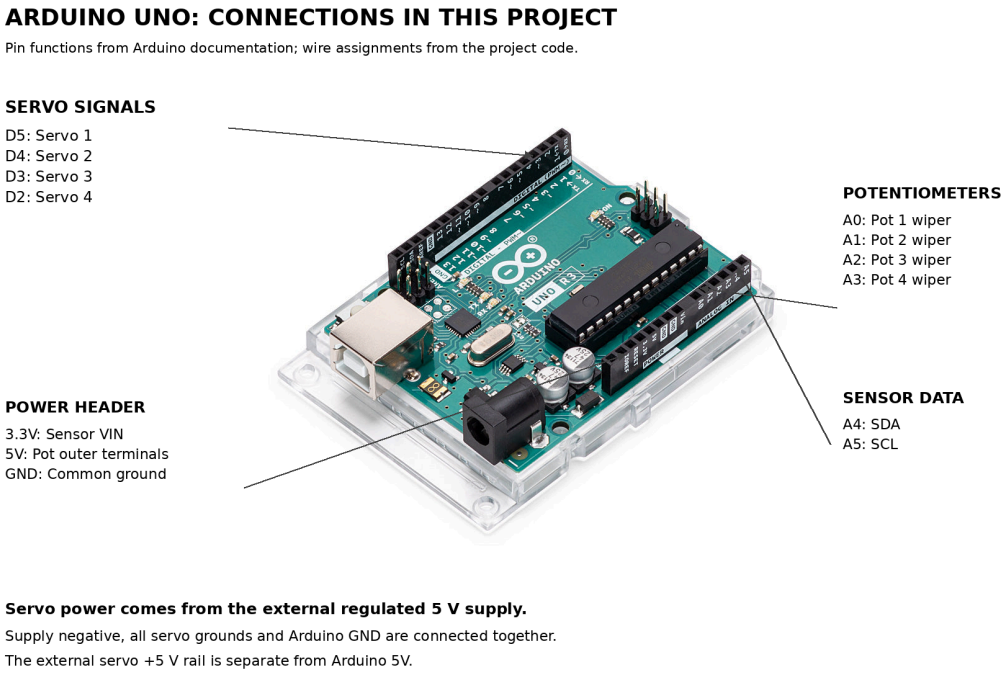


Figure 1. Official Arduino board photograph with the confirmed project connections. The sensor supply is 3.3 V, not 5 V.

4. Explanation: Potentiometers

A potentiometer is an adjustable resistor with three terminals. The two outer terminals connect to the ends of its resistance track. The centre terminal is the wiper, which moves along that track when the shaft turns. In this project, each potentiometer is used as a voltage divider.

Terminal	Connection	Reason
Outer terminal 1	Arduino 5V	Provides the high end of the voltage range.
Centre terminal (wiper)	A0, A1, A2 or A3	Provides the changing voltage that Arduino measures.
Outer terminal 2	Arduino GND	Provides the low end of the voltage range.

The Uno ADC converts the input voltage into a 10-bit reading. With the default reference, about 0 V gives 0 and about 5 V gives 1023. There are 1024 possible codes, numbered 0-1023. This is an input reading, not an angle.

50 kOhm is the nominal resistance between the two outer terminals. Turning the shaft changes the wiper position, not this total nominal resistance. Swapping the two outer wires reverses the control direction.

Why average? Repeated readings may contain small fluctuations. The code averages eight readings and then applies a filter. A loose wiper or ground connection can still produce large incorrect changes.

Example

For Servo 4, A3 is mapped from 0-1023 to 10-60 degrees. A reading near 512 gives a target near 35 degrees before the movement threshold and gradual stepping are applied.

5. Explanation: Servo Motors

A positional servo contains a motor, gears, a position sensor and an internal control circuit. Its output shaft moves towards a position requested by a control pulse. The gripper and joints use this motion to move the arm.

Wire / terminal	Connection	Function
Signal (often orange/yellow)	Assigned Arduino digital pin	Carries the timed control pulses. It does not supply motor power.
Power (usually red)	External regulated +5 V	Supplies the motor and its internal control circuit.
Ground (usually brown/black)	Common GND	Completes the power circuit and provides a signal reference.

Verify wire colors for the actual motor model. The assembly photographs show micro servos and larger motor housings, so one exact model specification should not be assumed for all four motors.

Servo	Signal pin	Pot input	Command limits
1 (gripper)	D5	A0	47-110 degrees
2	D4	A1	100-180 degrees
3	D3	A2	70-110 degrees
4	D2	A3	10-60 degrees

Why external power? Several loaded motors can require more current than the board can provide. The external supply negative must connect to Arduino GND so the signal voltage has a common reference.

Command versus movement: Servo.write() requests a position. The printed angle is this command, not a measurement of the physical joint. A stalled, unpowered or damaged motor may not reach it. A servo may also move at startup to match the current potentiometer setting.

6. Explanation: TCS34725 Color Sensor

The TCS34725 measures red, green, blue and clear-channel light intensity. Its infrared filter reduces the effect of infrared light. The clear channel measures a broad light level; it is not a color name. The program decides the name after reading the sensor.

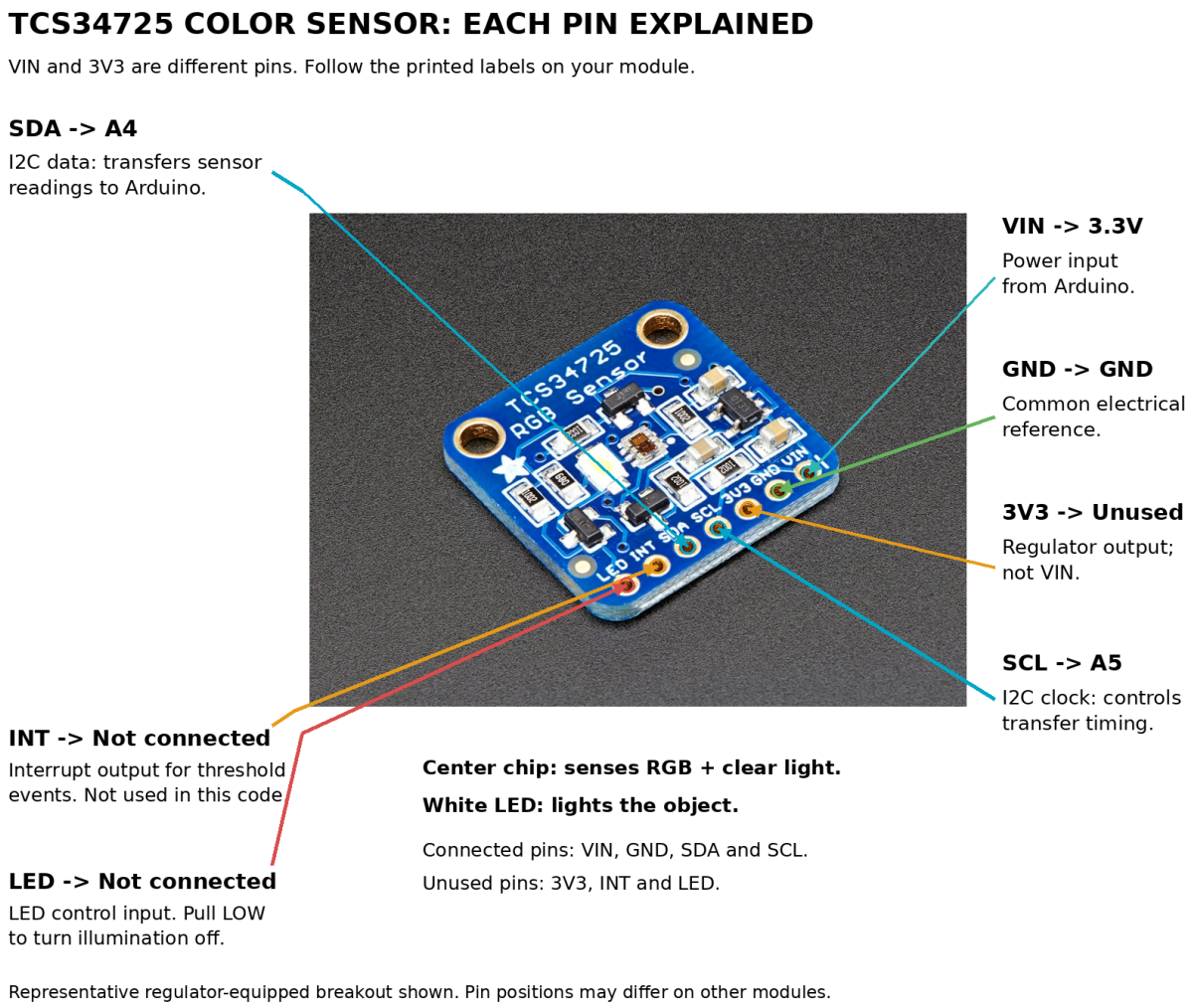
Sensor pin	Connection in this project	Function
VIN	Arduino 3.3V	Supplies the sensor breakout. VIN means voltage input on this module.
GND	Arduino GND	Provides the common electrical reference.
SDA	Arduino A4	Transfers I2C data between Arduino and the sensor.
SCL	Arduino A5	Carries the I2C clock generated by the controller.
3V3	Not connected	Regulator output on the common regulator-equipped breakout. It is different from VIN.
INT	Not connected	Interrupt output for configured threshold events; unused by this code.
LED	Not connected	Illumination control on the common breakout. Pulling it LOW turns its LED off. The sketch does not control this pin.

Why A4 and A5? The sensor uses I2C, a digital communication bus. Uno A4 has the SDA function and A5 has the SCL function. SDA carries data; SCL sets its communication timing. Sensor address 0x29 selects this device on the bus.

The VIN-to-3.3V connection describes the actual project. Other breakout versions may have different power circuits; the Adafruit wiring example uses 5 V at VIN. The bare sensor chip and a breakout module must not be treated as the same circuit.

Measurement settings: The code selects the library 50MS integration setting and 4x gain. The nominal integration setting is approximately 50 ms; it is separate from the 500 ms interval between scans.

Color Sensor: Pin Guide



Each arrow points to one labelled pin on the sensor breakout.

The four wires used in this project are:

VIN to Arduino 3.3V; GND to GND; SDA to A4; SCL to A5.

3V3, INT and LED are not connected in this project.

SDA carries data. SCL provides the clock for I2C communication.

VIN receives power. GND provides the common electrical reference.

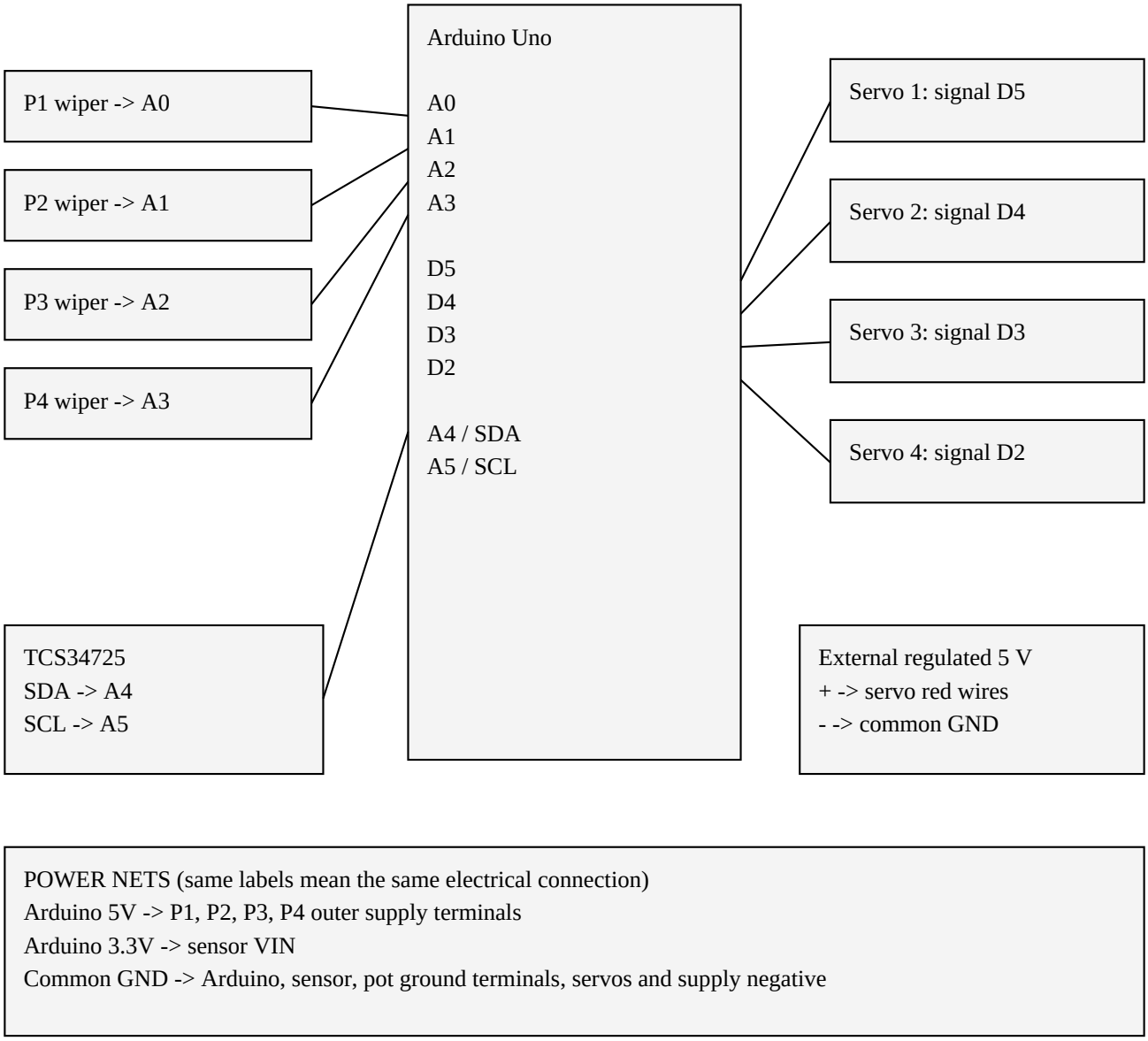
7. Wiring and Circuit Diagram

Use this table with the diagram on the next page. All mappings match the submitted manual-control code.

Arduino connection	Destination
A0 / A1 / A2 / A3	Wipers of potentiometers 1 / 2 / 3 / 4.
D5 / D4 / D3 / D2	Signal wires of servos 1 / 2 / 3 / 4.
5V	One outer terminal of all four potentiometers.
3.3V	TCS34725 VIN.
A4	TCS34725 SDA.
A5	TCS34725 SCL.
GND	Pot ground terminals, sensor GND, all servo grounds and external supply negative.
External supply +5 V	All four servo power wires.
USB	Computer connection for uploading and serial output.

The external servo +5 V rail and Arduino 5V rail are separate. Their grounds are connected. Do not route motor current through an Arduino signal pin. Breadboard rails can have a break at the centre; rail continuity should be checked rather than assumed.

Circuit Diagram: Signal and Power Nets



Motor +5 V and Arduino 5V are separate positive supply nets.

Figure 2. Logical wiring diagram. Power connections are given as named nets to keep the drawing readable. The digital pins carry signals only. Each potentiometer has one wiper, one 5 V terminal and one ground terminal.

8. Algorithm

Initialization - runs once

1. Start serial communication at 9600 baud.

2. Read all four potentiometers. Map each reading to its servo limits.

3. Store each initial angle and target, then attach the matching servo output.

4. Start I2C and configure a 25 ms Wire timeout if supported.

5. Initialize the color sensor and record the start time.

Main loop - repeats

6. Check whether at least 40 ms has passed since the last movement update.

7. If yes, read each pot using an eight-sample average. Update its filtered value.

8. Map the filtered value to an angle. Change the target only when the difference is at least 3 degrees.

9. If the current command differs from the target, step 1 degree towards the target and call Servo.write().

10. Check whether at least 500 ms has passed since the last scan. If not, return from this loop() call and repeat.

11. Read or retry the sensor. During the first 3 seconds, collect valid background samples. After that, compare readings with the background and stored color profiles.

12. Print all four commanded angles and the color/status. Repeat from step 6.

This is manual joint control with color measurement. There is no automatic grasping, sorting or pick-and-place sequence in this code.

9. Flowchart: Main Program

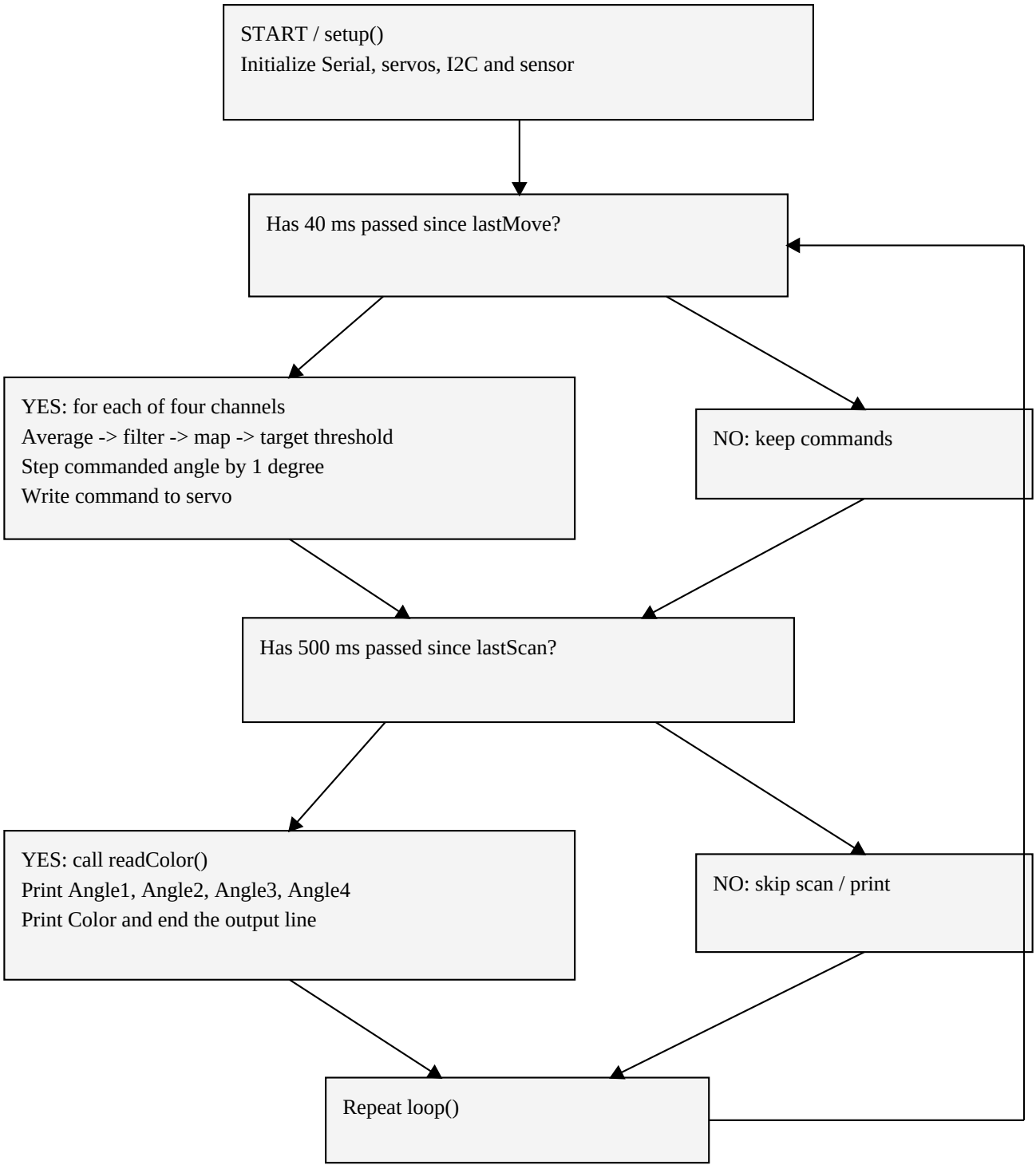


Figure 3. Main program flow. Timing checks avoid a long delay() in the main loop. Sensor reads and serial output can still take time.

Flowchart: Color Reading

readColor(): checks are performed in this order.

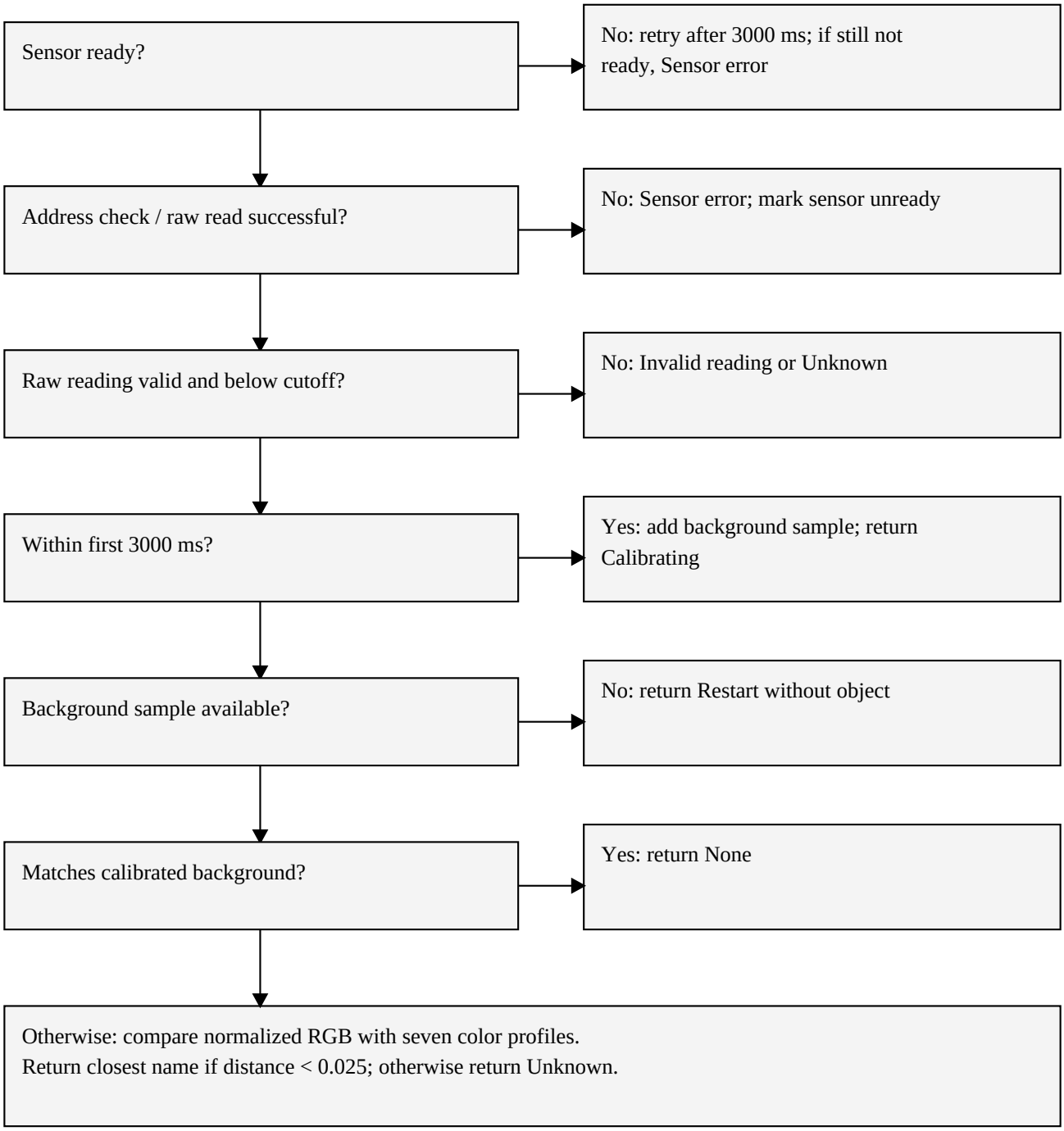


Figure 4. Each return ends readColor() for the current scan. The next scheduled scan calls it again. A failed startup calibration window is not automatically restarted by this sketch.

10. Timing and Output

Code setting	Meaning
40 ms	Minimum spacing between movement updates. At most one degree is added or subtracted per update.
500 ms	Minimum scheduled spacing between color scans and output lines.
3000 ms calibration	The background collection window begins after setup sensor initialization. Only valid readings are included.
3000 ms retry	Minimum interval before trying sensor initialization again after a recorded failure.
25000 microseconds	Wire timeout setting: 25 ms, if the installed core supports it.
50MS / 4X	Sensor integration and gain settings.
0.12	Filter coefficient: 12% new reading and 88% previous filtered value.
3 degrees	Threshold for accepting a new target.
0.0025 / 0.20	Background RGB-distance and clear-difference thresholds.
0.025	Maximum accepted color-profile squared distance.

These are program settings, not measured timing results. Sensor library calls and serial output can delay the next update. One degree per 40 ms gives a nominal command rate of 25 degrees per second when the loop keeps up. Physical speed also depends on the motor, load and power.

Example - format only

```
Angle1: 80 | Angle2: 140 | Angle3: 95 | Angle4: 35 | Color: Red
```

The numbers above are illustrative. The real output follows the potentiometers and the sensor. Serial Monitor must use 9600 baud.

Output Messages and Color Decisions

Color field	Meaning in this code
Calibrating	A valid startup sample is being added to the background.
None	The current reading is sufficiently similar to the startup background.
Red / Orange / Yellow / Green / Blue / Purple / White	The nearest stored profile passed the distance threshold.
Unknown	The raw-value cutoff was reached, or no profile was close enough.
Invalid reading	Clear is zero or the RGB sum is below 10.
Sensor error	Initialization or I2C communication failed, or a supported timeout was detected.
Restart without object	No valid background sample was captured during the startup window.

How color matching works

The code divides R, G and B by R+G+B. This gives three proportions whose sum is approximately 1. It compares them with each reference by adding the squared channel differences. The smallest distance is the nearest profile.

For example, (0.62, 0.25, 0.13) is exactly the current Orange reference, so its distance to that profile is zero. Real objects rarely produce these exact values. Most profiles are estimates and need calibration with the actual objects.

None is a background comparison, not a general object detector. An object that looks like the background may also return None. Moving the sensor or changing the lighting can change the background. Blue can still be reported incorrectly if the scene resembles its stored profile.

11. Hardware Photos

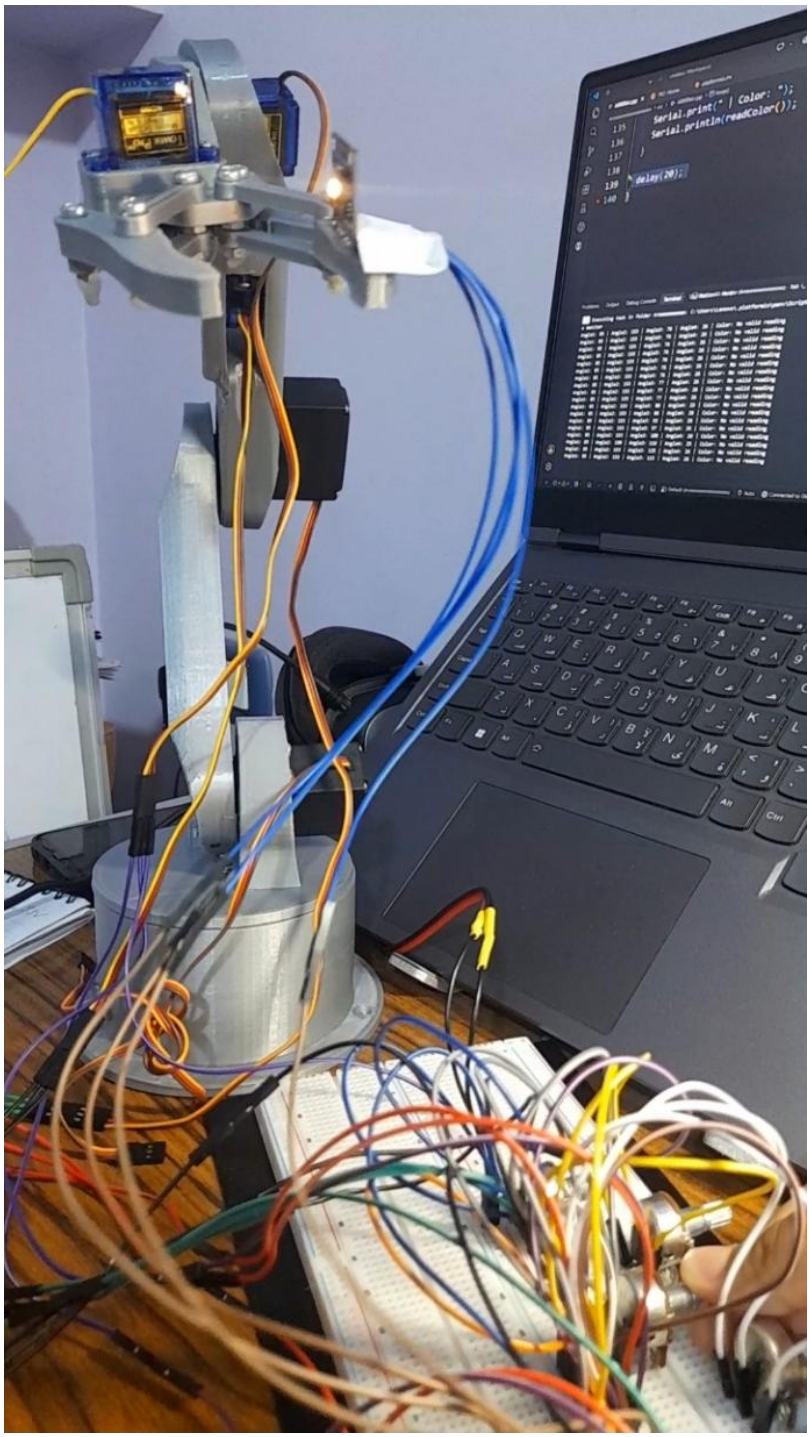
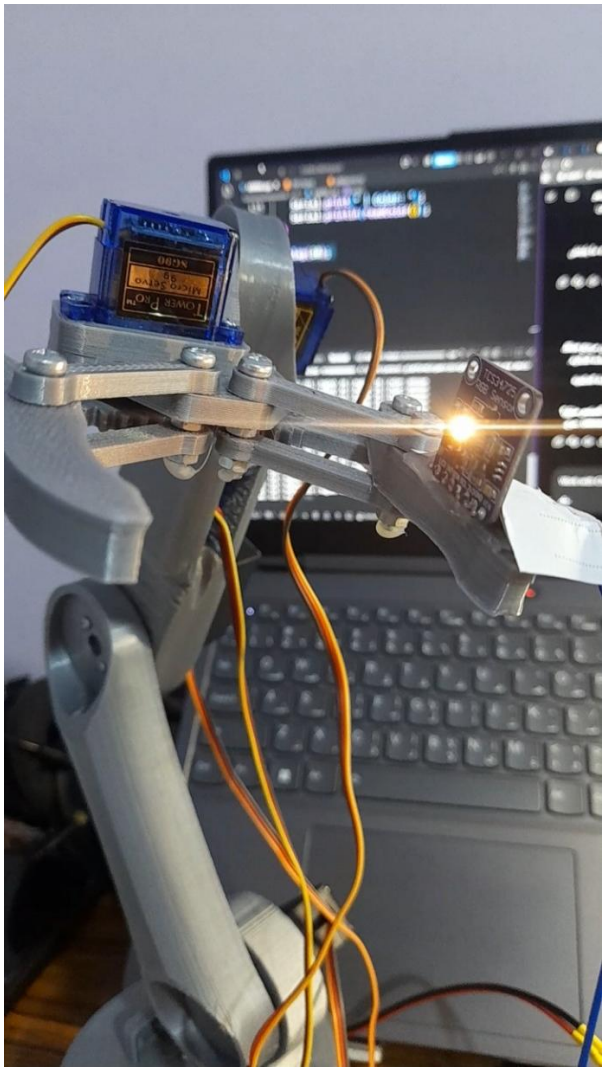


Figure 5. The physical robotic arm, control wiring and potentiometers. Photograph supplied by the project owner.

Hardware Photos: Sensor and Controls



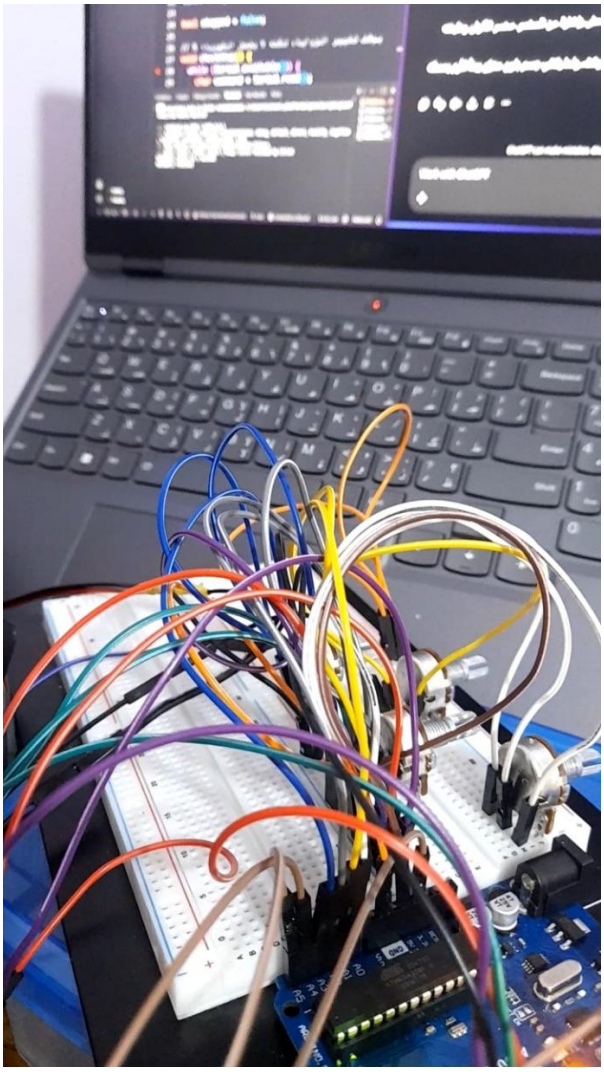


Figure 6 (left). The sensor is mounted near the gripper, with its illumination LED visible.

Figure 7 (right). Arduino Uno, breadboard, potentiometers and jumper connections. The wiring table gives the exact pin assignments because individual wires cannot all be traced reliably in these photographs.

12. Complete Code

This is the submitted manual-control program, with the same statements and settings. Blank-line spacing has been reduced. Line numbers in the next section refer to this listing and are not part of the uploadable code.

```
001  #include <Arduino.h>
002  #include <Servo.h>
003  #include <Wire.h>
004  #include <SPI.h>
005  #include <Adafruit_TCS34725.h>
006  Servo servos[4];
007  const byte servoPins[] = {5, 4, 3, 2};
008  const byte potPins[] = {A0, A1, A2, A3};
009  const int minAngle[] = {47, 100, 70, 10};
010  const int maxAngle[] = {110, 180, 110, 60};
011  int angles[4], targets[4];
012  float filtered[4];
013  unsigned long lastMove = 0, lastScan = 0;
014  unsigned long startTime = 0, lastRetry = 0;
015  Adafruit_TCS34725 tcs(
016    TCS34725_INTEGRATIONTIME_50MS,
017    TCS34725_GAIN_4X
018  );
019  bool sensorReady = false;
020  float background[4] = {};
021  byte backgroundSamples = 0;
022  struct Color {
023    const char* name;
024    float r, g, b;
025  };
026  const Color colors[] = {
027    {"Red",    .65, .23, .12},
028    {"Orange", .62, .25, .13},
029    {"Yellow", .45, .45, .10},
030    {"Green",  .41, .42, .17},
031    {"Blue",   .28, .33, .39},
032    {"Purple", .40, .23, .37},
033    {"White",  .33, .34, .33}
034  };
035  int readPot(byte index) {
036    analogRead(potPins[index]);
037    long total = 0;
038    for (byte i = 0; i < 8; i++) {
039      total += analogRead(potPins[index]);
040    }
041    return total / 8;
042  }
043  const char* sensorError() {
044    sensorReady = false;
045    lastRetry = millis();
046    return "Sensor error";
047  }
048  const char* readColor() {
049    if (!sensorReady && millis() - lastRetry >= 3000) {
```

Complete Code (continued)

```
050      lastRetry = millis();
051      sensorReady = tcs.begin();
052    }
053    if (!sensorReady) return "Sensor error";
054    Wire.beginTransmission(0x29);
055    if (Wire.endTransmission() != 0) return sensorError();
056    uint16_t r = 0, g = 0, b = 0, c = 0;
057  #ifdef WIRE_HAS_TIMEOUT
058    Wire.clearWireTimeoutFlag();
059  #endif
060    tcs.getRawData(&r, &g, &b, &c);
061  #ifdef WIRE_HAS_TIMEOUT
062    if (Wire.getWireTimeoutFlag()) return sensorError();
063  #endif
064    float total = (float)r + g + b;
065    if (!c || total < 10) return "Invalid reading";
066    if (c >= 20000 || r >= 20000 ||
067        g >= 20000 || b >= 20000) return "Unknown";
068    float data[] = {
069      r / total, g / total, b / total, (float)c
070    };
071    if (millis() - startTime < 3000) {
072      for (byte i = 0; i < 4; i++) background[i] += data[i];
073      backgroundSamples++;
074      return "Calibrating";
075    }
076    if (!backgroundSamples) return "Restart without object";
077    float dr = data[0] - background[0] / backgroundSamples;
078    float dg = data[1] - background[1] / backgroundSamples;
079    float db = data[2] - background[2] / backgroundSamples;
080    float baseC = background[3] / backgroundSamples;
081    float dc = abs(data[3] - baseC) / (data[3] + baseC + 1);
082    if (dr * dr + dg * dg + db * db < .0025f && dc < .20f)
083      return "None";
084    byte best = 0;
085    float nearest = 1000;
086    for (byte i = 0; i < 7; i++) {
087      dr = data[0] - colors[i].r;
088      dg = data[1] - colors[i].g;
089      db = data[2] - colors[i].b;
090      float distance = dr * dr + dg * dg + db * db;
091      if (distance < nearest) {
092        nearest = distance;
093        best = i;
094      }
095    }
096    return nearest < .025f ? colors[best].name : "Unknown";
097  }
098  void setup() {
```

Complete Code (continued)

```
  Serial.begin(9600);
  for (byte i = 0; i < 4; i++) {
    filtered[i] = readPot(i);
    angles[i] = targets[i] = map(
      (int)filtered[i], 0, 1023, minAngle[i], maxAngle[i]
    );
    servos[i].write(angles[i]);
    servos[i].attach(servoPins[i]);
  }
  Wire.begin();
#ifdef WIRE_HAS_TIMEOUT
  Wire.setWireTimeout(25000, true);
#endif
  sensorReady = tcs.begin();
  lastRetry = startTime = millis();
}
void loop() {
  if (millis() - lastMove >= 40) {
    lastMove = millis();
    for (byte i = 0; i < 4; i++) {
      filtered[i] += .12f * (readPot(i) - filtered[i]);
      int next = map(
        (int)filtered[i], 0, 1023, minAngle[i], maxAngle[i]
      );
      if (abs(next - targets[i]) >= 3) targets[i] = next;
      if (angles[i] == targets[i]) continue;
      angles[i] += targets[i] > angles[i] ? 1 : -1;
      servos[i].write(angles[i]);
    }
  }
  if (millis() - lastScan < 500) return;
  lastScan = millis();
  const char* color = readColor();
  for (byte i = 0; i < 4; i++) {
    Serial.print("Angle");
    Serial.print(i + 1);
    Serial.print(": ");
    Serial.print(angles[i]);
    Serial.print(" | ");
  }
  Serial.print("Color: ");
  Serial.println(color);
}
```

13. Code Explanation: Every Line

Read the numbered source listing first, then use this table while explaining the program. Each row describes one source line, including closing braces and continued statements. This section explains the existing code; it does not claim that the program has been tested on the physical arm.

Line	Code	Explanation
1	`#include <Arduino.h>`	Loads the Arduino core declarations, including analogRead(), millis() and Serial. Explicitly useful in a PlatformIO .cpp file.
2	`#include <Servo.h>`	Loads the Servo library, which generates the control pulses for the motors.
3	`#include <Wire.h>`	Loads the Wire library for I2C communication with the sensor.
4	`#include <SPI.h>`	Loads SPI declarations needed by some library dependencies. This sensor still communicates through I2C, not SPI.
5	`#include <Adafruit_TCS34725.h>`	Loads the Adafruit driver for the TCS34725 color sensor.
6	`Servo servos[4];`	Creates an array of four Servo objects. Their indexes are 0, 1, 2 and 3.
7	`const byte servoPins[] = {5, 4, 3, 2};`	Stores the four signal pins. Index 0 uses D5; index 3 uses D2. const means these values are not changed.
8	`const byte potPins[] = {A0, A1, A2, A3};`	Stores the four analog inputs in the same order as the servos.
9	`const int minAngle[] = {47, 100, 70, 10};`	Stores the minimum command for each servo, in degrees. These are project settings, not universal motor limits.
10	`const int maxAngle[] = {110, 180, 110, 60};`	Stores the maximum command for each servo. Each maximum belongs to the matching array index.
11	`int angles[4], targets[4];`	Creates two integer arrays. angles stores the latest commanded position; targets stores the desired position.
12	`float filtered[4];`	Creates four decimal values that hold the filtered potentiometer readings.
13	`unsigned long lastMove = 0, lastScan = 0;`	Stores the times of the last movement update and last sensor scan. unsigned long matches the millis() result.
14	`unsigned long startTime = 0, lastRetry = 0;`	Stores the start of background calibration and the last sensor initialization attempt.
15	`Adafruit_TCS34725 tcs(`	Creates the sensor object named tcs. The following two lines set its measurement options.

Line	Code	Explanation
16	`TCS34725_INTEGRATIONTIME_50MS,`	Selects the library integration setting named 50MS. It lets the sensor collect light over a short measurement period.
17	`TCS34725_GAIN_4X`	Selects a gain of 4x. Gain increases sensitivity; too much light or gain can saturate a reading.
18	`);`	Ends the sensor constructor call.
19	`bool sensorReady = false;`	Records whether sensor initialization has succeeded. It starts as false.
20	`float background[4] = {};`	Creates four zero-filled sums: normalized red, green, blue, and raw clear intensity.
21	`byte backgroundSamples = 0;`	Counts the valid background samples. byte is an unsigned 8-bit type on this board.
22	`struct Color {`	Defines a custom data type called Color that groups a name and three proportions.
23	`const char* name;`	Stores a pointer to the color name text. The pointed-to characters are read-only.
24	`float r, g, b;`	Stores the reference red, green and blue proportions as decimal numbers.
25	`};`	Closes the structure definition. The semicolon is required.
26	`const Color colors[] = {`	Creates a read-only array of seven reference profiles. These are project estimates, not official standard color values.
27	`{"Red", .65, .23, .12},`	Red reference: 65% red, 23% green, 12% blue of the RGB sum.
28	`{"Orange", .62, .25, .13},`	Orange reference: 62% red, 25% green, 13% blue. It is close to the red profile, so calibration matters.
29	`{"Yellow", .45, .45, .10},`	Yellow reference: 45% red, 45% green, 10% blue.
30	`{"Green", .41, .42, .17},`	Green reference: 41% red, 42% green, 17% blue.
31	`{"Blue", .28, .33, .39},`	Blue reference: 28% red, 33% green, 39% blue. A background can have similar proportions.
32	`{"Purple", .40, .23, .37},`	Purple reference: 40% red, 23% green, 37% blue.
33	`{"White", .33, .34, .33}`	White reference: 33% red, 34% green, 33% blue.
34	`};`	Closes the reference array. Each profile has a name and three values.
35	`int readPot(byte index) {`	Defines a function that returns an integer average for one potentiometer. index selects which input to read.

Line	Code	Explanation
36	`analogRead(potPins[index]);`	Reads and discards one conversion after selecting the channel. This can help the analog input settle, but does not guarantee noise-free input.
37	`long total = 0;`	Initializes the sample sum. long provides plenty of space for the added readings.
38	`for (byte i = 0; i < 8; i++) {`	Repeats the next statement eight times. i starts at 0 and ends at 7.
39	`total += analogRead(potPins[index]);`	Reads the selected analog pin and adds its value to total.
40	`}`	Ends the sampling loop.
41	`return total / 8;`	Divides the sum by eight and returns the average. Integer division drops any fractional part.
42	`}`	Ends readPot().
43	`const char* sensorError() {`	Defines a helper that returns a pointer to status text.
44	`sensorReady = false;`	Marks the sensor as not ready so the next scans can retry initialization.
45	`lastRetry = millis();`	Saves the error time. The next retry must wait at least 3000 ms.
46	`return "Sensor error";`	Returns the text printed in the Color field. It does not identify the physical cause of the fault.
47	`}`	Ends sensorError().
48	`const char* readColor() {`	Defines the function that returns a color name or sensor status.
49	`if (!sensorReady && millis() - lastRetry >= 3000) {`	Checks two conditions: the sensor is not ready AND at least 3000 ms has passed since the last attempt.
50	`lastRetry = millis();`	Saves the time before attempting initialization again.
51	`sensorReady = tcs.begin();`	Tries to detect and initialize the sensor. begin() returns true on success or false on failure.
52	`}`	Ends the retry condition.
53	`if (!sensorReady) return "Sensor error";`	If initialization has still failed, leaves this function immediately with Sensor error.
54	`Wire.beginTransmission(0x29);`	Prepares a transmission to I2C address 0x29, the TCS34725 address. 0x means hexadecimal.
55	`if (Wire.endTransmission() != 0) return sensorError();`	Ends the address check. A nonzero result means the transfer did not succeed; call the error helper.

Line	Code	Explanation
56	`uint16_t r = 0, g = 0, b = 0, c = 0;`	Creates four unsigned 16-bit variables for raw red, green, blue and clear data. Their initial values are zero.
57	`#ifdef WIRE_HAS_TIMEOUT`	Compile the following timeout code only if this Wire implementation defines WIRE_HAS_TIMEOUT.
58	`Wire.clearWireTimeoutFlag();`	Clears an older timeout flag before the raw sensor read.
59	`#endif`	Ends this conditional-compilation block.
60	`tcs.getRawData(&r, &g, &b, &c);`	Reads RGBC values. Each & passes the address of a variable so the library can write the result into it.
61	`#ifdef WIRE_HAS_TIMEOUT`	Again checks at compile time whether timeout support exists.
62	`if (Wire.getWireTimeoutFlag()) return sensorError();`	If an I2C timeout occurred, marks the sensor unready and returns Sensor error.
63	`#endif`	Ends the conditional-compilation block.
64	`float total = (float)r + g + b;`	Adds the three RGB channels. Casting r to float makes the sum use floating-point arithmetic.
65	`if (!c || total < 10) return "Invalid reading";`	Returns Invalid reading if clear is zero OR the RGB sum is below 10. This prevents division by zero later.
66	`if (c >= 20000 || r >= 20000 ||`	Starts a high-value rejection test for clear and red. The next source line continues the same condition.
67	`g >= 20000 || b >= 20000) return "Unknown";`	Completes the green and blue tests. Any channel at or above 20000 returns Unknown. This is a project cutoff, not a universal saturation formula.
68	`float data[] = {`	Creates a four-value array for the current sample.
69	`r / total, g / total, b / total, (float)c`	Normalizes each RGB channel by their sum, and stores clear as a float without normalizing it.
70	`};`	Closes the current data array.
71	`if (millis() - startTime < 3000) {`	Checks whether the first 3000 ms background window is still open.
72	`for (byte i = 0; i < 4; i++) background[i] += data[i];`	Adds all four current values to the corresponding background sums.
73	`backgroundSamples++;`	Increases the number of valid background samples by one.
74	`return "Calibrating";`	Returns Calibrating while the background reference is being collected.
75	`}`	Ends the startup calibration branch.

Line	Code	Explanation
76	`if (!backgroundSamples) return "Restart without object";`	If no valid sample was collected during the startup window, asks the user to restart without an object. It does not automatically reopen that window.
77	`float dr = data[0] - background[0] / backgroundSamples;`	Computes the difference between the current red proportion and the average background red proportion.
78	`float dg = data[1] - background[1] / backgroundSamples;`	Computes the corresponding green difference.
79	`float db = data[2] - background[2] / backgroundSamples;`	Computes the corresponding blue difference.
80	`float baseC = background[3] / backgroundSamples;`	Calculates the average background clear intensity.
81	`float dc = abs(data[3] - baseC) / (data[3] + baseC + 1);`	Calculates a relative clear-intensity difference. abs() removes the sign; +1 keeps the denominator nonzero.
82	`if (dr * dr + dg * dg + db * db < .0025f && dc < .20f)`	Checks that the squared RGB difference is below 0.0025 AND the relative clear difference is below 0.20. f marks a float literal.
83	`return "None";`	If both background tests pass, returns None. This means background-like, not guaranteed absence of an object.
84	`byte best = 0;`	Initializes the best matching profile index to the first profile.
85	`float nearest = 1000;`	Starts with a large distance so the first calculated profile distance can replace it.
86	`for (byte i = 0; i < 7; i++) {`	Checks all seven stored profiles, using indexes 0 through 6.
87	`dr = data[0] - colors[i].r;`	Subtracts the selected reference red proportion from the measured red proportion.
88	`dg = data[1] - colors[i].g;`	Subtracts the selected reference green proportion.
89	`db = data[2] - colors[i].b;`	Subtracts the selected reference blue proportion.
90	`float distance = dr * dr + dg * dg + db * db;`	Adds the squared differences. This is squared Euclidean distance: smaller means a closer match. No square root is needed for comparison.
91	`if (distance < nearest) {`	Checks whether this profile is closer than every profile checked so far.
92	`nearest = distance;`	Saves the new smallest distance.
93	`best = i;`	Saves the index of that profile.
94	`}`	Ends the better-match condition.

Line	Code	Explanation
95	`}`	Ends the seven-profile loop.
96	`return nearest < .025f ? colors[best].name : "Unknown";`	Uses the ternary operator: return the best name if distance is below 0.025; otherwise return Unknown. The cutoff is a project setting.
97	`}`	Ends readColor().
98	`void setup() {`	Defines setup(), which runs once after startup or reset. void means it returns no value.
99	`Serial.begin(9600);`	Starts serial communication at 9600 bits per second. Set Serial Monitor to the same rate.
100	`for (byte i = 0; i < 4; i++) {`	Initializes all four servo channels one after another.
101	`filtered[i] = readPot(i);`	Reads the current potentiometer position into the initial filtered value.
102	`angles[i] = targets[i] = map(`	Starts one map() expression and assigns its result to both the current command and the target.
103	`(int)filtered[i], 0, 1023, minAngle[i], maxAngle[i]`	Converts the reading to an integer and maps 0-1023 into that servo's configured angle range.
104	`);`	Closes the map() call and finishes the assignment.
105	`servos[i].write(angles[i]);`	Stores the initial angle before attaching the output. There is no servo position feedback; the physical joint may still move at startup.
106	`servos[i].attach(servoPins[i]);`	Attaches this Servo object to its selected digital output pin and enables its control pulses.
107	`}`	Ends the four-channel setup loop.
108	`Wire.begin();`	Initializes the Uno I2C interface, using A4 for SDA and A5 for SCL.
109	`#ifdef WIRE_HAS_TIMEOUT`	Includes the timeout configuration only when the Wire core supports it.
110	`Wire.setWireTimeout(25000, true);`	Sets a Wire timeout of 25000 microseconds, or 25 ms. true requests a Wire hardware reset on timeout; it is not a power reset of the sensor.
111	`#endif`	Ends this conditional-compilation block.
112	`sensorReady = tcs.begin();`	Attempts the first sensor initialization and saves whether it succeeded.
113	`lastRetry = startTime = millis();`	Sets both timestamp variables to the current time, starting the calibration window.

Line	Code	Explanation
114	`}`	Ends setup().
115	`void loop() {`	Defines loop(), which is called repeatedly after setup().
116	`if (millis() - lastMove >= 40) {`	Runs the movement block only when at least 40 ms has passed since the previous update.
117	`lastMove = millis();`	Records the time of this update. This is a minimum interval, not an exact real-time deadline.
118	`for (byte i = 0; i < 4; i++) {`	Processes each of the four channels.
119	`filtered[i] += .12f * (readPot(i) - filtered[i]);`	Adds 12% of the difference between the new average and the previous filtered value. Equivalently: 0.88 x old + 0.12 x new.
120	`int next = map(`	Starts the expression that converts the filtered reading to a candidate angle called next.
121	`(int)filtered[i], 0, 1023, minAngle[i], maxAngle[i]`	Uses the same analog limits and per-servo angle limits as in setup().
122	`);`	Closes the map() call.
123	`if (abs(next - targets[i]) >= 3) targets[i] = next;`	Accepts the new target only if it differs from the previous target by at least 3 degrees. Small changes are ignored.
124	`if (angles[i] == targets[i]) continue;`	If this servo is already at its target command, skips the remaining statements for this servo and moves to the next index.
125	`angles[i] += targets[i] > angles[i] ? 1 : -1;`	Adds 1 degree if the target is larger, or subtracts 1 degree if it is smaller. This limits command changes per update.
126	`servos[i].write(angles[i]);`	Sends the updated angle command to that Servo object.
127	`}`	Ends the channel loop.
128	`}`	Ends the timed movement block.
129	`if (millis() - lastScan < 500) return;`	If fewer than 500 ms have passed, ends this loop() call. Arduino calls loop() again; the servo pulses remain active.
130	`lastScan = millis();`	Records the time of the new scan.
131	`const char* color = readColor();`	Calls readColor() and stores the returned text pointer. It may hold a color name or a status message.
132	`for (byte i = 0; i < 4; i++) {`	Starts printing the four commanded angles.
133	`Serial.print("Angle");`	Prints the word Angle without starting a new line.

Line	Code	Explanation
134	`Serial.print(i + 1);`	Prints i+1, giving human-readable labels 1 through 4 instead of array indexes 0 through 3.
135	`Serial.print(": ");`	Prints a colon and a space.
136	`Serial.print(angles[i]);`	Prints the latest integer angle command for this channel.
137	`Serial.print(" | ");`	Prints a separator between output fields.
138	`}`	Ends the angle-printing loop.
139	`Serial.print("Color: ");`	Prints the label for the color field.
140	`Serial.println(color);`	Prints the color/status text and ends the output line.
141	`}`	Ends loop(). Arduino starts the next call automatically.

14. Important Symbols and Terms

Symbol / term	Simple meaning
const	The value is not meant to change after initialization.
byte / int / long	Integer types. byte holds small nonnegative values; long provides a larger integer range.
unsigned long	Nonnegative integer type used for millis() timestamps.
float / .12f	Decimal type; f marks a floating-point literal.
bool	A true or false value.
[]	Declares an array or selects an array element. Indexing starts at zero.
{}	Groups a block of statements or provides initial values.
;	Ends a declaration or statement.
= / ==	Assignment / equality comparison.
!= / >= / <	Not equal / greater than or equal / less than.
! / && / ||	NOT / AND / OR.
+= / ++	Add and assign / increase by one.
&r	Address of r, passed so the library can store a result in r.
const char*	Pointer to read-only text, such as a color name.
condition ? a : b	Choose a when the condition is true, otherwise choose b.
return / continue	Leave the current function / skip to the next loop iteration.
#ifdef / #endif	Include a block at compile time only when a named definition exists.
(int) / (float)	Convert a value to a selected numeric type.

15. Conclusion

This project uses an Arduino Uno to control four servo motors manually through four potentiometers. A TCS34725 sensor adds color measurement near the gripper. The Serial Monitor shows the four angle commands and the current color or sensor status.

The program uses averaging, filtering and gradual command changes to reduce the effect of small input changes. Stable motor power, secure connections and suitable mechanical limits are still needed. The color profiles are approximate and should be calibrated for the actual objects and lighting.

The project demonstrates analog inputs, digital control pulses, I2C communication, external motor power and serial output in one circuit.